class attributes:

it belongs to the class itself they will be shared by all the instances. such attributes are defined in the class body statements usually at the top, for eligibility.

Example:

```
class sample:
   count=0#class attribute
   def increase(self):
      sample.count+=1
s=sample()
s.increase()
print(s.count)#1
s1=sample()
s1.increase()
print(s1.count)#2
print(sample.count)#2
```

instance attributes:

→these are not shared by objects.

→every object has its own copy of the instance attributes

1.var():

it displays the attribute of an instance in the form of an dictionary.

2.dir():

it displays more attributes than various functions, as it is not limited to instead to instances.

it displays the class attributes as well

Example:

```
class stu:
   def __init__(self):#default constructor
      self.name="pooja"
      self.branch="CSE"
   def display(self):
      print(self.name)
      print(self.branch)
s1=stu()
print("dictionary form:",vars(s1))
print(dir(s1))
```

DECORATORS

in python, functions are first class objects.
→functions are objects, they can be referenced to, passed to a variable and returned from other function as well.
→function can be defined in another function and can also passed as argument to another function.
Definition of decorator:

- **it is very powerful tool in python.**
- **it allows programmers to modify the behavior of functions and classes.**
- **it allows us wrap another function in order to extend the behavior of wrapped function without permanently modifying it.**
- **here, functions are taken as arguments into another function and then called inside the wrapper function.**

program:

```
#find out execution time of function using decorator
import time
import math
def cal_time(func):
   def inner(*args,**kwargs):
      begin=time.time()
      func(*args,**kwargs)
      end=time.time()
      print("time taken:",func.__name__,end-begin)
   return inner
@cal_time#wrapped function
def fact(num):
   time.sleep(4)
   print(math.factorial(num))
fact(9)
```

class methods:
it receives the class as implicit first argument, just like an instance method receives the instance.
Syntax:

```
class class_name:
    @classmethod
```

```
    def fun_name(cls,rag1,arg2……):
      ……
      ……..
```

→A class method is a method which is bound to the class and not the object of the class.
→they have the access to the state of the class as it takes as class parameter that points to the class and not the object instance.
→it can modify a class state that would apply across all the instances of the class.
→if you want to call class method, we can call with object_name.method_name.

static method:

it does not receive an implicit argument.
Syntax:

```
class class_name:
  @staticmethod
  def fun_name(arg1,arg2…….):
      …….
      ……
```

→it cannot accesses or modify the class state.
→it is present in a class it makes sense for the method to be present in the class.
→by using static method you need not create object to the class name but we can call
class_name.method_name()

Example:

```
from datetime import date
class Person:
  def __init__(self,name,age):
    self.name=name
    self.age=age
  @classmethod
  def frombirthyear(cls,name,year):
    return cls(name,date.today().year-year)
  @staticmethod
  def isadult(age):
    return age>18
p1=Person('john',23)
p2=Person.frombirthyear('mani',1990)
print(p1.age)
print(p2.age)
print(Person.isadult(25))
print(Person.isadult(15))
```

interview question:

What is the difference between class method and static method?

Class method	Static method
It takes class as first parameter	No specific parameter
It can access or modify class state	It can't access or modify it
We use @classmethod decorator in python to create class method	We can use @staticmethod decorator to create static method
We can use class method to create factory methods. factory methods returns class object(similar to constructor) for different use cases	We generally use static methods to create utility functions

How we can define private methods in python?

In python, there is no existence of private methods that cannot be accessed except inside a class. however, to define a private method prefix the member name with double underscore.

Note:

the __init__ method is a constructor and runs as soon as object of a class is instantiated.

Example:

```
class Base:
   def fun(self):#public method
      print("public method")
   def __fun(self):#private method
      print("private function")
class derived(Base):
   def __init__(self):
      Base.__init__(self)
   def call_public(self):
      print("\n inside derived class")
      self.fun()
   def call_private(self):
      self.__fun()
obj1=Base()
obj1.fun()
obj2=derived()
obj2.call_public()
obj2.call_private()
```